



Ruby and Ruby on rails Course

Table of Contents

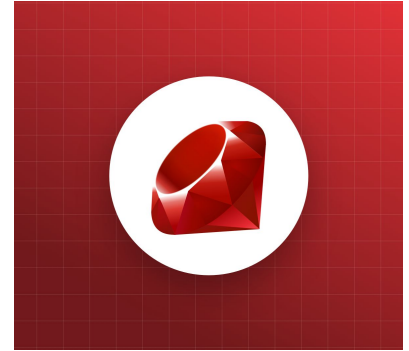
- **Introduction**
- **Installation of ruby**
- **Basics of ruby language**
- **Type conversion vs kernel methods**
- **Variables**
- **Methods, Flow control, Loops**
- **Data structures**
 - ◆ **Arrays**
 - ◆ **Hashes**
- **Code block**
- **More Iterators**
- **Exception handling**
- **Files**
- **Classes, modules, mixins**

Introduction

Do we still need to know the syntax?

Introduction: What is Ruby Language?

- Object-oriented language
- Interpreted programming language.
- Created by Yukihiro Matsumoto in 1993.
- Ruby was quickly adopted in Japan and then expanded to the rest of the world.
- It gained popularity when the Ruby on Rails framework was released.
- Ruby is still widely used in big companies like Shopify, Airbnb, and Github, which also contribute to the source code and investment and language development.



How to install Ruby on your machine?

Preferable to install it using version manager like **rbenv**, **rvm (ruby)**
Or ASDF (version manager for all languages)

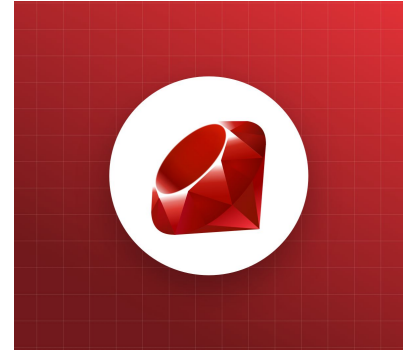
In the course we will use rbenv

Linux:

- Follow this [link](#) for installation

Windows:

- Download Ruby Installer from this link [RubyInstaller](#),
- After installation, open PowerShell or CMD and run:
C:/Ruby24-x64/bin/ridk.cmd



First program

- Open your editor and type the following:

```
puts 'Hello, world!'
```

- Save the file as program.rb to the documents directory, then open Terminal and type:

```
ruby ~/Documents/program.rb
```

- The result should be printed to the terminal!

Hello, world!

Documentation

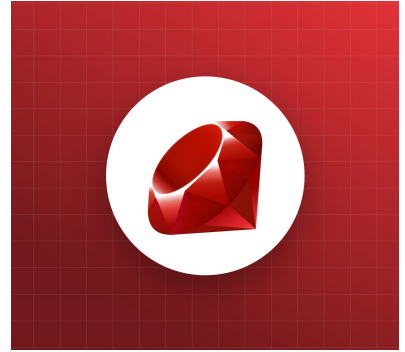
- You can download HTML version of the documentation via <http://www.ruby-doc.org>
- You can use ri tool (ruby index) to browse documentation locally

```
ri Array
```

```
ri Array.sort
```

```
ri Hash#each
```

- You can also use offline documentation browsers, like [Dash](#), [Velocity](#) and [Zeal](#)



Pry

To play with ruby locally, it's recommended to use **pry** (interactive ruby shell) to test and evaluate ruby statements and expressions.

- Open Terminal and type:

```
pry  
pry(main):001:0>
```

- If it's not installed, install it by typing:

```
gem install pry
```


Basics of ruby language

Comments

```
# This is a comment
```

```
=begin
```

```
This is a multi-line comment.
```

```
The beginning line must start with "=begin"  
and the ending line must start with "=end".
```

```
You can do this, or start each line in a  
multi-line comment with the # character.
```

```
=end
```

Basics of ruby language

- Ruby is a strongly object-oriented language.
- Absolutely everything in Ruby is an object.

Numbers

```
1 + 1 ==> 2
8 - 1 ==> 7
10 * 2 ==> 20
35 / 5 ==> 7
2 ** 5 ==> 32
5 % 3 ==> 2
```

Basics of ruby language

Numbers

#In Ruby, numbers without decimal points are called integers, and numbers with decimal points are usually called floating-point numbers or, more simply, floats.

```
puts 1 + 2          #=> 3
```

```
puts 2 * 3          #=> 6
```

Integer division

When you do arithmetic with integers, you'll get integer answers

```
puts 3 / 2          #=> 1
```

```
puts 3.0 / 2         #=> 1.5
```

```
puts 10 - 11         #=> -1
```

```
puts 1.5 / 2.6       #=> 0.5769230769230769
```

Basics of ruby language

Numbers

The increment and decrement operators (++ and --) are not available in Ruby, However, do note that the += and -= are available.

```
i += 1  
i -= 1
```

#Underscores may be inserted into integer literals (though not at the beginning or end), and this feature is sometimes used as a thousands separator

```
1_000_000_000
```

Basics of ruby language

Equality & Inequality

```
1 == 1 #=> true  
2 == 1 #=> false
```

```
1 != 1 #=> false  
2 != 1 #=> true
```

Basics of ruby language

Comparisons

```
1 < 10 #=> true  
1 > 10 #=> false
```

```
2 <= 2 #=> true  
2 >= 2 #=> true
```

Basics of ruby language

Combined comparison

Combined comparison operator `<=>`:
returns ``1`` > the first argument is greater.
returns ``1`` > when the second argument is greater,
returns ``0`` > otherwise

```
1 <=> 10 #=> -1 (1 < 10)
10 <=> 1  #=> 1  (10 > 1)
1 <=> 1   #=> 0  (1 == 1)
```

Basics of ruby language

Falsey values

Apart from false itself, nil is the only other 'falsey' value

nil —> false

false —> false

0 —> true

"" —> true

[] —> true

Basics of ruby language

Strings

Can use " or ' for Strings, but ' is more Efficient

```
puts 'Hello, World!'  
puts "Hello, World!"
```

Basics of ruby language

Strings

```
placeholder = 'use string interpolation'
```

```
"I can #{placeholder} when using double quoted strings"  
#=> "I can use string interpolation when using double quoted strings"
```

You can combine strings using `+`, but not with other types

```
'hello ' + 'world'  #=> "hello world"  
'hello ' + 3        #=> TypeError: can't convert Fixnum into String  
'hello ' + 3.to_s   #=> "hello 3"  
"hello #{3}"         #=> "hello 3"
```

Basics of ruby language

Strings

Combine strings and operators

```
'hello ' * 3      #=> "hello hello hello "
```

Append to string (Strings are mutable)

```
'hello' << ' world'  #=> "hello world"
```

You can print to the output with a newline at the end

```
puts "I'm printing!"  #=> I'm printing!  
#=>nil
```

Print to the output without a newline

```
print "I'm printing!"  #=> "I'm printing!" =>nil
```

Basics of ruby language

Strings

```
a = "hello world"  
a.concat(33)
```

`#=> "hello world!"`

```
a[1]  
a[2, 3]  
a[2..3]  
"hello".capitalize  
a.capitalize!
```

`#=> "e"`

`#=> "llo"`

`#=> "ll"`

`#=> "Hello"`

`#=> "Hello world!"`

```
"hello".include? "lo"  
"hello".index('e')  
"abcd".insert(0, 'X')
```

`#=> true`

`#=> 1`

`#=> "Xabcd"`

Basics of ruby language

Strings: Escape characters

```
\\ #=> Need a backslash in your string?  
\b #=> Backspace  
\r #=> Carriage return, for those of you that love typewriters  
\n #=> Newline. You'll likely use this one the most.  
\s #=> Space  
\t #=> Tab  
\" #=> Double quotation mark  
' #=> Single quotation mark
```

Basics of ruby language

Symbol

Symbols are immutable, reusable constants represented internally by an integer value.

```
:pending.class      #=> Symbol  
status = :pending
```

Strings can be converted into symbols and vice versa.

```
status.to_s         #=> "pending"  
"argon".to_sym      #=> :argon
```

Basics of ruby language

Symbol vs String

Strings will return different object id in memory although its the same value.

```
puts "string".object_id  
puts "string".object_id
```

Symbol will return same object id in memory.

```
puts :symbol.object_id  
puts :symbol.object_id
```

Basics of ruby language

Range

Sequences have a start point, an end point, and a way to produce successive values in the sequence

```
(1..10).class           #=> Range
(1..10).to_a            #=> [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
(1...10).to_a           #=> [1, 2, 3, 4, 5, 6, 7, 8, 9]
digits = -1..8
puts digits.include?(5)  #=> true
puts digits.min          #=> -1
puts digits.max          #=> 8
puts digits.reject {|ele| ele < 5 }  #=> [5, 6, 7, 8]
```


Basics of ruby language

Regular Expressions

```
[0-9].class      #=> Regexp  
m1 = /Ruby/.match("The future is Ruby")  
puts m1.class    #=> MatchData  
m2 = "The future is Ruby" =~ /Ruby/  
puts m2         #=> 14
```

Basics of ruby language

NIL

nil is an actual object. You can call methods on nil, just like any other object.

You can add methods to nil, just like any other object.

```
puts nil.class # NilClass
```

Type conversion vs Kernel Methods

Feature	Type Conversion Methods (to_i, to_f, to_s, etc.)	Kernel Methods (Integer(), Float(), String(), etc.)
Return Value on Failure	Typically return a default value (e.g., 0, 0.0, "")	Raise an error (ArgumentError or TypeError)
Use Case	Safe for transformations where a default value is an acceptable fallback	Preferred when input validation is required, and non-convertible values should trigger an error
Error Handling	Silent failure; may lead to subtle bugs if not used carefully	Explicit failure; forces handling of exceptional cases, making code more robust

Variables

```
x = 25
```

```
#=> 25
```

```
x
```

```
#=> 25
```

Note that assignment returns the value assigned.

This means you can do multiple assignment.

```
x = y = 10
```

```
#=> 10
```

By convention, use snake_case for variable names.

```
snake_case = true
```

```
#=> true
```

Variables

```
x, y = 5, 7
```

```
x      #=> 5
```

```
y      #=> 7
```

will interchange the values of x and y.

```
x, y = y, x
```

```
x      #=> 7
```

```
y      #=> 5
```

Variables

Heredoc style is available

```
a = <<END STR  
This is the string  
And a second line  
END_STR
```

```
puts a
```

Variables

Variables types

Variables				Constants and Class Names
Local	Global	Instance	Class	
name	\$debug	@name	@@total	PI
fish_and_chips	\$CUSTOMER	@point_1	@@symtab	FeetPerMile
x_axis	\$_	@X	@@N	String
thx1138	\$plan9	@_	@@x_pos	MyClass
_26	\$Global	@plan9	@@SINGLE	JazzSong

Method definition

define the method with param

```
def say_goodnight(name)
  # notice that we don't need a return keyword
  Puts "hello"
  "Good night, " + name
end
```

Calling the method

```
puts say_goodnight("Islam")  #=> "Good night, Islam"
```

Or

```
puts say_goodnight "Islam"  #=> "Good night, Islam"
```


Methods

Method alias

```
def oldmtd  
  "old method"  
end
```

```
alias same_old_mthd oldmtd
```

```
def oldmtd  
  "old improved method"  
end
```

```
puts oldmtd  
puts same_old_mthd
```

Methods

Pass by Value vs Reference

```
def downer(string)
  string.downcase
end
```

```
a = "HELLO"
downer(a)      #=> "hello"
puts a        #=> "HELLO"
```

```
class << a
  def downer
    self.replace downcase
  end
end
```

Methods

Bang (!) Methods

#Ruby methods that modify an object in-place and end in an exclamation mark (!) are known as bang methods. Also called dangerous methods!

```
a = 'ISLAM'
a.downcase
a
a.downcase!
a
```

```
#=> "ISLAM"
#=> "islam"
#=> "ISLAM"
#=> "islam"
#=> "islam"
```

Methods

Boolean (?) Methods

Any method whose name ends with ? returns a Boolean value

```
puts "Islam".include?"s"  
puts "Islam".include?"z"
```

#=> true

#=> false

Flow control

If-elsif-else-unless

if i want to add multiple conditions i can use keyword <or> or || , <and> or &&

```
if (true || false) && true
  'if statement'
elsif false
  'else if, optional'
else
  'else, also optional'
end
x = 5
p 5 if x == 5
p 'not 10' unless x == 10
```

Flow control

Ternary Operator

(condition) ? (result if condition is true) : (result if condition is false)

example

```
age = 15  
puts (13...19).include?(age) ? "teenager" : "not a teenager"  
#=> "teenager"
```

Flow control

Case

```
year = 2000
```

you can remove “then” keyword if you put the result in next line

```
def isLeap(year)
```

```
  leap = case
```

```
    when year % 400 == 0 then true
```

```
    when year % 100 == 0 then false
```

```
    else year % 4 == 0
```

```
  end
```

```
  leap
```

```
end
```

Flow control

Operators (Equality)

Operator	Description	Use Case	Example	Result
<code>==</code>	Checks if the values of two operands are equal, focusing on the value rather than the type.	General comparisons where type coercion might apply.	<code>5 == 5.0</code>	<i>true</i>
<code>===</code>	Used primarily in case statements to check a range of values or types. Its behavior varies by operand type.	When determining if an object falls within a range or belongs to a certain type/class.	<code>(1..5) === 3</code>	<i>true</i>
<code>eq?</code>	Checks if both value and type are the same. Used by Hash to compare keys.	When a distinction between types is important, such as in hash key comparisons.	<code>5.eql?(5.0)</code>	<i>false</i>
<code>equal?</code>	Checks if two operands refer to the exact same object in memory.	When you need to know if two variables point to the same object.	<code>a = "ruby"; b = a; a.equal?(b)</code>	<i>true</i>

Loops

while

```
while weight < 100 and num_pallets <= 30  
    # code to be executed  
end
```

```
square = 2  
square = square*square while square < 1000
```

Loops

for

```
for a in 1..5 do  
  # code to be executed  
end
```

```
arr = ["GFG", "G4G", "Geeks", "Sudo"]  
# using for loop  
for ele in arr do  
  puts ele  
end
```

Loops

loop

```
loop do  
  # code to be executed  
  break if Boolean_Expression  
end
```

times

```
5.times do  
  puts "anything"  
end
```

Arrays

This is an array.

```
array = [1, 2, 3, 4, 5] #=> [1, 2, 3, 4, 5]
```

Arrays can contain different types of items.

```
[1, 'hello', false] #=> [1, "hello", false]
```

%w(foo bar) is a shortcut for ["foo", "bar"]. Meaning it's a notation to **write an array of strings** separated by spaces instead of commas and without quotes around them.

```
%w(foo bar baz) #=> ["foo", "bar", "baz"]
```

Arrays

Arrays can be indexed.

From the front...

```
arr = [1, 2, 3, 4, 5]
```

```
arr[0]           #=> 1
```

```
arr.first        #=> 1
```

```
arr[12]          #=> nil
```

or from the back...

```
arr[-1]          #=> 5
```

```
arr.last         #=> 5
```

or with a start index and length...

```
arr[2, 3]         #=> [3, 4, 5]
```

Arrays

...or with a range...

```
arr = [1, 2, 3, 4, 5]
```

```
arr[1..3]      #=> [2, 3, 4]
```

Like arithmetic, [var] access is just syntactic sugar for calling a method '[]' on an object.

```
arr[] 0        #=> 1
```

```
arr[] 12       #=> nil
```

You can add to an array...

```
arr << 6       #=> [1, 2, 3, 4, 5, 6]
```

Or like this

```
arr.push(6)    #=> [1, 2, 3, 4, 5, 6]
```

Arrays

Array methods

```
newarr = [45, 23, 1, 90]
```

sort method would sort it in asc order how to make it in decs ?

```
newarr.sort           #=> [1, 23, 45, 90]
```

```
newarr.length         #=> 4
```

```
newarr.include?(23)  #=> true
```

Parallel Assignment

```
a = 1, 2, 3           #=> [1, 2, 3]
```

```
a == [1, 2, 3]        #=> true
```

```
b = [1, 2, 3]         #=> [1, 2, 3]
```

```
b == [1, 2, 3]        #=> true
```

```
a, b = 1, 2, 3        => a == 1, b == 2
```

```
c, = 1, 2, 3, 4       => c == 1
```

Hash

Hashes are Ruby's primary dictionary with key/value pairs.

Hashes are denoted with curly braces.

```
hash = { 'color' => 'green', 'number' => 5 }
```

```
hash.keys      #=> ['color', 'number']
```

Hashes can be quickly looked up by key.

```
hash['color']   #=> "green"
```

```
hash['number']  #=> 5
```

Asking a hash for a key that doesn't exist returns nil.

```
hash['nothing here'] #=> nil
```


Hash

When using symbols for keys in a hash, you can use an alternate syntax.

```
hash = { :defcon => 3, :action => true }
```

```
hash.keys           #=> [:defcon, :action]
```

```
hash = { defcon: 3, action: true } ⇒ no space between name of symbol and colon
```

```
hash.keys           #=> [:defcon, :action]
```

Check existence of keys and values in hash

```
hash.key?(:defcon)  #=> true
```

```
hash.value?(3)      #=> true
```

Code block

Are chunks of code between braces or between do- end.

Can associate with method invocations, almost as if they were parameters

Ruby standard is to use braces for single line blocks and do- end for multi-line blocks.

```
x = 10
5.times do |x|
  puts "x inside the block: #{x}"
end
```

Can be done like that:

```
5.times{|x|
  puts "x inside the block: #{x}"
}
puts "x outside the block: #{x}"    #=> 10
```

Code block

pass code block to function to execute it

```
def call_block
  puts "Start of method"
  yield
  yield
  puts "End of method"
end
call_block { puts "In the block" }
```

Code block

pass parameter to the code block

```
def call_block
  yield("hello", 99)
end
call_block {|str, num|
  print str , " your number " , num.to_s
}
```

#=> hello your number 99

More Iterators

methods that return successive elements from some kind of collection, such as an array.

work mainly with blocks

```
animals = %w(ant bee cat dog)
animals.each {|animal| puts animal }
```

#for ... in

```
for ele in ['fee', 'fi', 'fo', 'fum']
  print ele, " "
end
```

More Iterators

```
['cat', 'dog', 'horse'].each {|name| print name, " " }  
5.times { print "*" }  
3.upto(6) {|i| print i }  
6.downto(3) {|i| print i }  
('a'..'e').each {|char| print char }
```

Exception handling

```
begin
  # Code here that might raise an exception
  raise NoMemoryError, 'You ran out of memory.'
  raise RuntimeError, ''
rescue NoMemoryError => exception_variable
  puts 'NoMemoryError was raised', exception_variable
rescue RuntimeError => other_exception_variable
  puts 'RuntimeError was raised now'
else
  puts 'This runs if no exceptions were thrown at all'
ensure
  puts 'This code always runs no matter what'
end
```

[Useful article about creating your own exception](#)

Files

Read/Write Text Files

Open and read from a text file

Note that since a block is given, file will automatically be closed when the block terminates

```
File.open('test.rb', 'r') do |f1|  
  while line = f1.gets  
    puts line  
  end  
end
```

Create a new file and write to it

```
File.open('test.rb', 'w') do |f2|  
  # use "\n" for two lines of text  
  f2.puts "Created by Islam\nThank God!"  
end
```


Files

Including Other Files

The load method includes the named Ruby source file **every time** the method is executed:

```
load 'filename.rb'
```

The more commonly used require method loads any given file **only once**:

```
require 'filename.rb'
```

There are another method called `Kernel#require_relative`

[Check this article to know the difference](#)

Classes

Class definition

```
class Dog
  def initialize(name)
    # Instance variable
    @name = name
  end

  def bark
    puts 'Ruff! Ruff!'
  end
end
Dog.new('scary')
```

Classes

Inheritance

```
class Person
  def initialize(name, age, job)
    @name = name
    @age = age
    @job = job
  end
end
```

```
class Student < Person
  def initialize(name, age, job, branch)
    super(name, age, job)
    @branch = branch
  end
end
```

Classes

Redefining Methods

```
class OR
  def mtd
    puts "First definition of method mtd"
  end
  def mtd
    puts "Second definition of method mtd"
  end
end
```

```
OR.new.mtd
```

Classes

Methods Overriding

```
class A
  def a
    puts 'In class A'
  end
end
```

```
class B < A
  def a
    puts 'In class B'
  end
end
```

```
b = B.new
b.a
```

Classes

Methods Overloading

```
class Rectangle
  def initialize(*args)
    if args.size < 2 || args.size > 3
      puts 'This method takes either 2 or 3 arguments'
    else
      if args.size == 2
        puts 'Two arguments'
      else
        puts 'Three arguments'
      end
    end
  end
end

Rectangle.new([10, 23], 4, 10)
Rectangle.new([10, 23], [14, 13])
```

Classes

Class Variables

```
class Student < Person
  @@student_no = 0
  def initialize(name, age, job, branch)
    super(name, age, job)
    @branch = branch
    self.increment
  end
  def increment
    @@student_no += 1
  end
end
```

Classes

Class Methods

```
class Example
  def instance_method # instance method
  end
  def self.class_method # class method
  end
end
```


Classes

Method Missing

```
class Dummy
  def method missing(m, *args, &block)
    puts "There's no method called #{m} here - please try again."
  end
end
```

```
Dummy.new.anything
```

Classes

Access control

```
class MyClass  
  def method1 # default is 'public'  
end
```

```
protected # subsequent methods will be 'protected'  
def method2 # will be 'protected'  
end
```

```
private # subsequent methods will be 'private'  
def method3 # will be 'private'  
end
```

```
public # subsequent methods will be 'public'  
def method4 # and this will be 'public'  
end  
end
```

Classes

Access control

by default, the instance variables are **private** to expose them you need setters and getters using `attr_accessor`. [For more info](#)

```
class Song
  def initialize(name, artist)
    @name= name
    @artist= artist
  end
  def name
    @name
  end
  def name=(name)
    @name = name
  end
end
```

Modules/Mixins

Module definition

```
module Rails
  class Application
    # ...
  end
end
app = Rails::Application.new
```

Mixins

```
module Greeting
  def hello
    puts 'Hello Sir'
  end
end
```



```
class Person
  include Greeting
end
```



```
Person.new.hello
```